

**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
ADVANCED COLLEGE OF ENGINEERING AND MANAGEMENT
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
KALANKI, KATHMANDU**



**[CT755]
A PROJECT REPORT ON
“ELECTRONICS COMPONENT STORE MANAGEMENT”**

Submitted By:
Antika Nepal ACE082BCT018

Project Supervisor:
Mukunda Raj Joshi

3rd August 2026

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my project supervisor, MRJ, for the valuable guidance, encouragement, and support provided throughout the development of this project, “Electronics Component Store Management”. The insightful suggestions and constructive feedback given during every stage of this project helped me understand the practical application of Object Oriented Programming concepts in solving a real-world inventory management problem.

I am also thankful to the Department of Electronics and Computer Engineering, Advanced College of Engineering and Management, for providing the opportunity and necessary environment to carry out this project as part of the Object Oriented Programming course. I extend my thanks to my classmates and seniors who assisted me by sharing their knowledge and experience during the design, coding, and testing phases of this project.

Finally, I would like to thank my family for their continuous support and motivation throughout this project work.

Antika Nepal
ACE082BCT018

ABSTRACT

This project, “Electronics Component Store Management”, is a console-based inventory management system developed in C++ using the principles of Object Oriented Programming. The main objective of the project is to computerize the process of managing electronic components in a small store by allowing the shopkeeper to add, view, update, delete, and purchase components such as resistors, capacitors, inductors, diodes, LEDs, transistors, ICs, microcontrollers, and display modules. The system is built using classes and objects, where a base class `ElectronicComponent` stores the common attributes of a component (name, quantity, unit price, rating, and description) and provides input and display behaviour, while a derived class `InventoryItem` inherits from it using public inheritance and overrides the display function through runtime polymorphism, since the base class functions are declared virtual. All the inventory records are managed by a separate `Inventory` class using the C++ Standard Template Library vector, and the data is stored persistently on the disk in a text file, `components.txt`, using file handling so that the records are available even after the program is closed and reopened. The methodology followed for the project involved requirement analysis, class design, coding of each module (add, view, update, delete, purchase, save and load), and testing of the program with sample data through the console. The results obtained show that the system is able to correctly add new components, display them in tabular form, update and delete records by name, generate a purchase bill by reducing the available stock, and save and reload the inventory data without loss. The project concludes that Object Oriented Programming concepts such as classes, inheritance, virtual functions, encapsulation, and file handling can be effectively combined to build a simple and functional inventory management system, and it can further be enhanced in the future with graphical interfaces and database support.

Keywords: C++, encapsulation, file handling, inheritance, inventory management, object oriented programming, polymorphism

TABLE OF CONTENTS

ACKNOWLEDGMENT	I
ABSTRACT.....	II
TABLE OF CONTENTS	III
LIST OF FIGURES	V
LIST OF TABLES	VI
LIST OF ABBREVIATIONS	VII
CHAPTER 1: INTRODUCTION.....	1
1.1 Background.....	8
1.2 Motivation.....	8
1.3 Problem Statement.....	8
1.4 Objective	9
1.4.1 General Objective	9
1.4.2 Specific Objectives	9
1.5 Significance of the Study	9
CHAPTER 2: LITERATURE REVIEW	10
CHAPTER 3: METHODOLOGY	11
3.1 Class Design.....	11
3.2 Algorithm.....	12
3.3 Flowchart	14
3.4 Source Code	15
CHAPTER 4: RESULTS AND ANALYSIS.....	23
4.1 Gantt Chart.....	23
4.2 Results.....	23
4.2.1 Add Component	23
4.2.2 View Component	24
4.2.3 Update Component.....	24
4.2.4 Purchase Component.....	25
4.2.5 Save Component	25
4.2.6 Exit	26
CHAPTER 5: CONCLUSION, LIMITATIONS AND FUTURE ENHANCEMENT	27
5.1 Conclusion	27
5.2 Limitations	27
5.3 Future Enhancement	27
REFERENCES.....	28

LIST OF FIGURES

Fig 3.1: Flowchart of the Overall System (Tabular Representation).....	7
Fig 4.2.1 Add Component	23
Fig 4.2.2 View Component	24
Fig 4.2.3 Update Component.....	24
Fig 4.2.4 Purchase Component.....	25
Fig 4.2.5 Save Component	25
Fig 4.2.6 Exit	26

LIST OF TABLES

Table 3.1: Attributes of ElectronicComponent Class	4
Table 3.2: Menu Options of Inventory Class.....	5
Table 3.3: Flowchart in Tabular Form.....	7
Table 4.1: Gantt Chart of Project Schedule	9

LIST OF ABBREVIATIONS

IC	: Integrated Circuit
IDE	: Integrated Development Environment
LED	: Light Emitting Diode
OOP	: Object Oriented Programming
STL	: Standard Template Library

CHAPTER 1: INTRODUCTION

1.1 Background

Electronic component stores that deal with items such as resistors, capacitors, inductors, diodes, LEDs, transistors, ICs, microcontrollers, and display modules generally maintain their stock records manually in registers or spreadsheets. As the number of components and daily transactions increases, manually tracking the available quantity, price, and technical rating of every component becomes time consuming and error prone. A computerized system that stores this information in a structured way, allows quick searching and updating of records, and automatically calculates the bill during a purchase can significantly simplify the day-to-day operation of such a store.

This project, “Electronics Component Store Management”, has been developed as a console-based application using C++ and Object Oriented Programming concepts to address this need at a small scale, suitable for a single store or a laboratory store room.

1.2 Motivation

As a student of Electronics and Computer Engineering, handling and purchasing electronic components such as resistors, capacitors, and ICs for laboratory work is a regular activity. The motivation behind this project came from observing that small component stores and college laboratory stores often keep track of components using handwritten registers, which makes it difficult to quickly check the available stock, price, or rating of a component. This inspired the idea of developing a simple, menu-driven C++ application that can store, update, and manage electronic component records efficiently, while also applying and strengthening the Object Oriented Programming concepts learned in the course, such as classes, inheritance, virtual functions, and file handling.

1.3 Problem Statement

Manual record keeping of electronic components creates several problems: it is difficult to quickly search for a specific component among many handwritten entries, stock quantities are not updated instantly after every purchase, calculating the total bill for a purchase has to be done manually and is prone to arithmetic errors, and there is no permanent, easily reusable digital record of the components once the register is misplaced or damaged. These problems reduce the efficiency of the store and increase the chances of mistakes in billing and stock

tracking. Hence, there is a need for a lightweight, computerized inventory system that can add, search, update, delete, and bill components, and store this information persistently on disk.

1.4 Objective

1.4.1 General Objective

The general objective of this project is to design and implement a console-based Electronics Component Store Management System using Object Oriented Programming concepts in C++ that can manage the inventory of electronic components.

1.4.2 Specific Objectives

- To design classes that represent an electronic component and its inventory using encapsulation, inheritance, and virtual functions.
- To implement functionalities to add, view, update, and delete electronic components from the inventory.
- To implement a purchase module that reduces stock quantity and generates a purchase bill.
- To implement file handling so that inventory records are saved to and loaded from a text file, making the data persistent between program runs.

1.5 Significance of the Study

This project is significant because it demonstrates the practical application of core Object Oriented Programming concepts, namely classes and objects, inheritance, virtual functions and runtime polymorphism, encapsulation through getter and setter functions, and file handling, in solving a real-life inventory management problem. The beneficiaries of this system include small electronic component store owners and college laboratory stores, who can use a simplified version of this program to maintain accurate records of stock, avoid billing errors, and save time compared to manual record keeping. For the developer, the project builds a strong practical foundation in C++ programming, software design using classes, and persistent data storage using files, which are essential skills for further studies and software development work.

CHAPTER 2: LITERATURE REVIEW

Inventory management systems have been a widely studied and implemented application area in computer science because nearly every retail or store-based business needs to track the items it stocks, sells, and reorders. Traditional inventory systems were maintained manually using ledgers, which, as discussed in classic works on business data processing, suffer from problems of duplication, delay in updating records, and difficulty in generating accurate reports (Hart, 1998).

With the growth of procedural and later Object Oriented Programming languages, many small-scale inventory systems began to be implemented as standalone desktop or console applications. Object Oriented Programming, as described by early software engineering literature, allows a real-world entity such as a “product” or “component” to be modeled directly as a class with its own attributes and behaviour, which makes the resulting program easier to extend and maintain compared to a purely procedural approach (Shaw, 1995).

Several student and academic-level projects have implemented similar inventory or store management systems in C++ using classes to represent items, arrays or STL containers such as vector to hold multiple records, and simple file handling using ifstream and ofstream to store data between program executions, which is consistent with the general recommendations for structuring small-scale data-persistent applications (Webster and Watson, 2002).

This project builds upon these established practices from prior literature by combining a base class `ElectronicComponent` with a derived class `InventoryItem` connected through public inheritance and virtual functions, together with an Inventory management class that uses the C++ Standard Template Library vector container and text-file based persistence, to implement a lightweight component store management system tailored specifically to electronic components such as resistors, capacitors, and integrated circuits.

CHAPTER 3: METHODOLOGY

This chapter explains the way the Electronics Component Store Management System was designed and implemented. It covers the class design used in the C++ program, the algorithm followed by each major module, a tabular representation of the system flowchart, and the source code structure.

3.1 Class Design

The system is built around three classes, following an inheritance-based design:

- **ElectronicComponent** – the base class that stores the common attributes of any electronic component (type, name, quantity, unitPrice, rating, description) and provides virtual input() and display() functions, along with getter and setter functions for encapsulation.
- **InventoryItem** – a class publicly derived from ElectronicComponent. It calls the base class constructor to initialise the component's attributes and overrides the virtual display() function, demonstrating runtime polymorphism.
- **Inventory** – the management class that holds a vector of InventoryItem objects and implements the menu, addComponent(), viewComponent(), updateComponent(), deleteComponent(), purchaseComponent(), saveData(), and loadData() functions.

Table 3.1: Attributes of ElectronicComponent Class

Attribute	Data Type	Description
type	string	Category of the component, e.g. Resistor, Capacitor, IC
name	string	Name/label of the specific component
quantity	int	Number of units available in stock
unitPrice	float	Price of a single unit of the component
rating	string	Power rating or voltage rating of the component
description	string	Short description of the component

Table 3.2: Menu Options of Inventory Class

Option	Function Called	Purpose
1	addComponent()	Add a new electronic component to the inventory
2	viewComponent()	Display all components in a formatted table
3	updateComponent()	Search a component by name and update its details
4	deleteComponent()	Search a component by name and remove it from the vector
5	purchaseComponent()	Reduce stock of a component and print a purchase bill
6	saveData()	Save all current records to components.txt
7	saveData() then exit	Save data and terminate the program

3.2 Algorithm

Algorithm: Main Program

Step 1: Start.

Step 2: Create an object 'store' of class Inventory and call store.loadData() to load previously saved records from components.txt, if any.

Step 3: Display the menu with 7 options: Add, View, Update, Delete, Purchase, Save, Exit.

Step 4: Read the user's choice.

Step 5: If choice is 1, call addComponent(); if 2, call viewComponent(); if 3, call updateComponent(); if 4, call deleteComponent(); if 5, call purchaseComponent(); if 6, call saveData(); if 7, call saveData() and print 'Thank You'; otherwise print 'Invalid Choice'.

Step 6: Repeat Steps 3 to 5 while choice is not equal to 7.

Step 7: Stop.

Algorithm: addComponent()

Step 1: Display the 9 available component types (Resistor, Capacitor, Inductor, Diode, LED, Transistor, IC, Microcontroller, Display Module).

Step 2: Read the user's choice of component type.

Step 3: If the choice is not between 1 and 9, print 'Invalid Choice' and return.

Step 4: Set the type of the new InventoryItem object to the selected component type.

Step 5: Call item.input() to read the name, quantity, unit price, rating, and description from the user.

Step 6: Push the new item into the components vector using push_back().

Step 7: Print 'Component Added Successfully'.

Algorithm: viewComponent()

Step 1: If the components vector is empty, print 'No Components Found' and return.

Step 2: Print the table header (Type, Name, Qty, Price, Rating, Description).

Step 3: For each item in the components vector, call its display() function to print its details in tabular form.

Step 4: End.

Algorithm: updateComponent()

- Step 1: If the components vector is empty, print 'No Components Found' and return.
- Step 2: Read the name of the component to search for.
- Step 3: Search the vector linearly for an item whose name matches the search string.
- Step 4: If found, display its current details, then read new values for name, quantity, unit price, rating, and description, and update the object using the setter functions.
- Step 5: If not found after checking all items, print 'Component Not Found'.

Algorithm: deleteComponent()

- Step 1: If the components vector is empty, print 'No Components Found' and return.
- Step 2: Read the name of the component to delete.
- Step 3: Search the vector linearly for an item whose name matches the search string.
- Step 4: If found, remove it from the vector using `components.erase()` and print 'Component Deleted Successfully'.
- Step 5: If not found, print 'Component Not Found'.

Algorithm: purchaseComponent()

- Step 1: If the components vector is empty, print 'No Components Found' and return.
- Step 2: Read the name of the component to purchase and search for it in the vector.
- Step 3: If not found, print 'Component Not Found' and return.
- Step 4: If found, display the available stock and read the quantity to purchase.
- Step 5: If the requested quantity is less than or equal to zero, print 'Invalid Quantity' and return.
- Step 6: If the requested quantity is greater than the available stock, print 'Not Enough Stock Available' and return.
- Step 7: Otherwise, subtract the purchased quantity from the available stock and update it using `setQuantity()`.
- Step 8: Calculate the total bill as purchased quantity multiplied by unit price.
- Step 9: Print the purchase bill showing component name, type, purchased quantity, unit price, total bill, and remaining stock.

Algorithm: *saveData()* and *loadData()*

saveData(): Step 1: Open the file components.txt in output mode. Step 2: For every item in the components vector, write its type, name, quantity, unit price, rating, and description on separate lines. Step 3: Close the file and print 'Data Saved Successfully'.

loadData(): Step 1: Open the file components.txt in input mode. Step 2: If the file cannot be opened, return with an empty inventory. Step 3: Clear the existing components vector. Step 4: Repeatedly read six lines at a time (type, name, quantity, price, rating, description) and construct an InventoryItem from them, pushing it into the vector, until the end of file is reached. Step 5: Close the file.

3.3 Flowchart

Since this is a console-based menu-driven program, the overall control flow of the system is represented below in tabular form instead of a graphical flowchart.

Table 3.3: Flowchart of the System in Tabular Form

Step No.	Process / Decision	Next Step
1	Start the program	Go to Step 2
2	Load existing data from components.txt using <i>loadData()</i>	Go to Step 3
3	Display the main menu with 7 options	Go to Step 4
4	Read user's choice	Go to Step 5
5 (Decision)	Is choice equal to 1? → Yes: call <i>addComponent()</i> No: check next choice	Go to Step 6
6 (Decision)	Is choice equal to 2? → Yes: call <i>viewComponent()</i> No: check next choice	Go to Step 7
7 (Decision)	Is choice equal to 3? → Yes: call <i>updateComponent()</i> No: check next choice	Go to Step 8
8 (Decision)	Is choice equal to 4? → Yes: call <i>deleteComponent()</i> No: check next choice	Go to Step 9
9 (Decision)	Is choice equal to 5? → Yes: call <i>purchaseComponent()</i> No: check next choice	Go to Step 10
10 (Decision)	Is choice equal to 6? → Yes: call <i>saveData()</i> No: check next choice	Go to Step 11
11 (Decision)	Is choice equal to 7? → Yes: call <i>saveData()</i> , print 'Thank You' No: print 'Invalid Choice'	Go to Step 12
12 (Decision)	Is choice equal to 7?	If No, go back to Step 3; If Yes, go to Step 13
13	Stop the program	End

3.4 Source Code

The complete source code of the Electronics Component Store Management System, implemented in C++, is presented below.

```
#include<iostream>
#include<fstream>
#include<vector>
#include<iomanip>
#include<string>

using namespace std;

class ElectronicComponent
{
protected:

    string type;
    string name;
    int quantity;
    float unitPrice;
    string rating;
    string description;

public:
    ElectronicComponent()
    {
        type="";
        name="";
        quantity=0;
        unitPrice=0;
        rating="";
        description="";
    }

    ElectronicComponent(string t,string n,int q,float p,string r,string d)
    {
        type=t;
        name=n;
        quantity=q;
        unitPrice=p;
        rating=r;
        description=d;
    }

    virtual ~ElectronicComponent(){}
    virtual void input()
    {
        cin.ignore();
        cout<<"Component Name : ";
        getline(cin,name);
        cout<<"Quantity : ";
        cin>>quantity;
        cout<<"Unit Price : ";
        cin>>unitPrice;
        cin.ignore();

        cout<<"Power Rating / Voltage : ";
        getline(cin,rating);
        cout<<"Description : ";
        getline(cin,description);
    }

    virtual void display()
    {
        cout<<left
        <<setw(18)<<type
        <<setw(20)<<name
        <<setw(10)<<quantity
```

```

<<setw(12)<<unitPrice
<<setw(18)<<rating
<<description<<endl;
    }
    string getType()
    {
        return type;
    }
    string getName()
    {
        return name;
    }
    int getQuantity()
    {
        return quantity;
    }
    float getPrice()
    {
        return unitPrice;
    }
    string getRating()
    {
        return rating;
    }

    string getDescription()
    {
        return description;
    }
    void setType(string t)
    {
        type=t;
    }

    void setName(string n)
    {
        name=n;
    }
    void setQuantity(int q)
    {
        quantity=q;
    }
    void setPrice(float p)
    {
        unitPrice=p;
    }

    void setRating(string r)
    {
        rating=r;
    }

    void setDescription(string d)
    {
        description=d;
    }
};

class InventoryItem:public ElectronicComponent
{
public:
    InventoryItem(){}
    InventoryItem(string t,string n,int q,float p,string r,string d):
ElectronicComponent(t,n,q,p,r,d)
    {
    }
    void display() override
    {
        ElectronicComponent::display();
    }
};
class Inventory
{
private:

```



```

        vector<InventoryItem> components;

public:
    void menu();

    void addComponent();
    void viewComponent();
    void updateComponent();
    void deleteComponent();
    void purchaseComponent();
    void saveData();
    void loadData();
};

void Inventory::menu()
{
    cout<<"\tELECTRONICS COMPONENTS STORE MANAGEMENT\n\n";

    cout<<"1. Add Component"<<endl;
    cout<<"2. View Component"<<endl;
    cout<<"3. Update Component"<<endl;
    cout<<"4. Delete Component"<<endl;
    cout<<"5. Purchase Component"<<endl;
    cout<<"6. Save Data"<<endl;
    cout<<"7. Exit"<<endl;
}

void Inventory::addComponent()
{
    InventoryItem item;

    string componentType[] =
    {
        "Resistor",
        "Capacitor",
        "Inductor",
        "Diode",
        "LED",
        "Transistor",
        "IC",
        "Microcontroller",
        "Display Module"
    };

    int choice;

    cout<<"Select Component Type"<<endl;

    for(int i=0;i<9;i++)
    {
        cout<<i+1<<" . "<<componentType[i]<<endl;
    }

    cout<<"\nEnter Choice :";
    cin>>choice;

    if(choice<1 || choice>9)
    {
        cout<<"\nInvalid Choice!\n"<<endl;
        return;
    }

    item.setType(componentType[choice-1]);

    item.input();

    components.push_back(item);

    cout<<"\nComponent Added Successfully!\n"<<endl;
}

```

```

void Inventory::viewComponent()
{
    if(components.empty())
    {
        cout<<"\nNo Components Found!\n"<<endl;
        return;
    }

    cout<<left
    <<setw(18)<<"Type"
    <<setw(20)<<"Name"
    <<setw(10)<<"Qty"
    <<setw(12)<<"Price"
    <<setw(18)<<"Rating"
    <<"Description"<<endl;

    for(int i=0;i<components.size();i++)
    {
        components[i].display();
    }
}

void Inventory::updateComponent()
{
    if(components.empty())
    {
        cout<<"\nNo Components Found!\n"<<endl;
        return;
    }

    string search;

    cin.ignore();

    cout<<"\nEnter Component Name to Update : ";
    getline(cin,search);

    bool found=false;

    for(int i=0;i<components.size();i++)
    {
        if(components[i].getName()==search)
        {
            found=true;
            cout<<"\nCurrent Details\n";

            cout<<"Type          : "<<components[i].getType()<<endl;
            cout<<"Name          : "<<components[i].getName()<<endl;
            cout<<"Quantity       : "<<components[i].getQuantity()<<endl;
            cout<<"Unit Price    : "<<components[i].getPrice()<<endl;
            cout<<"Rating        : "<<components[i].getRating()<<endl;
            cout<<"Description   : "<<components[i].getDescription()<<endl;

            string newName,newRating,newDescription;
            int newQuantity;
            float newPrice;

            cout<<"\nEnter New Details\n";
            cout<<"Component Name : ";
            getline(cin,newName);
            cout<<"Quantity : ";
            cin>>newQuantity;
            cout<<"Unit Price : ";
            cin>>newPrice;
            cin.ignore();

```

```

        cout<<"Power Rating / Voltage : ";
        getline(cin,newRating);
        cout<<"Description : ";
        getline(cin,newDescription);

        components[i].setName(newName);
        components[i].setQuantity(newQuantity);
        components[i].setPrice(newPrice);
        components[i].setRating(newRating);
        components[i].setDescription(newDescription);

        cout<<"\nComponent Updated Successfully!\n"<<endl;
        break;
    }
}

if(!found)
{
    cout<<"\nComponent Not Found!\n"<<endl;
}
}

void Inventory::deleteComponent()
{
    if(components.empty())
    {
        cout<<"\nNo Components Found!\n"<<endl;
        return;
    }

    string search;

    cin.ignore();

    cout<<"\nEnter Component Name to Delete : ";
    getline(cin,search);

    bool found=false;

    for(int i=0;i<components.size();i++)
    {
        if(components[i].getName()==search)
        {
            components.erase(components.begin()+i);
            found=true;

            cout<<"\nComponent Deleted Successfully!\n"<<endl;
            break;
        }
    }
    if(!found)
    {
        cout<<"\nComponent Not Found!\n"<<endl;
    }
}

void Inventory::purchaseComponent()
{
    if(components.empty())
    {
        cout<<"\nNo Components Found!\n"<<endl;
        return;
    }

    string search;
    int buyQuantity;

    cin.ignore();

    cout<<"\nEnter Component Name to Purchase : ";
    getline(cin,search);

```

```

bool found=false;

for(int i=0;i<components.size();i++)
{
    if(components[i].getName()==search)
    {
        found=true;

        cout<<"\nAvailable Stock : "<<components[i].getQuantity();
        cout<<"\nEnter Quantity to Purchase : ";
        cin>>buyQuantity;
        if(buyQuantity<=0)
        {
            cout<<"\nInvalid Quantity!\n"<<endl;
            return;
        }
        if(buyQuantity>components[i].getQuantity())
        {
            cout<<"\nNot Enough Stock Available!"<<endl;
            return;
        }

        int remainingStock =
            components[i].getQuantity()-buyQuantity;

        components[i].setQuantity(remainingStock);

        float totalBill =
            buyQuantity*components[i].getPrice();

        cout<<"\tPURCHASE BILL";
        cout<<"Component Name : "
<<components[i].getName()<<endl;
        cout<<"Component Type : "
<<components[i].getType()<<endl;
        cout<<"Purchased Qty : "
<<buyQuantity<<endl;
        cout<<"Unit Price : Rs. "
<<components[i].getPrice()<<endl;
        cout<<"Total Bill : Rs. "
<<totalBill<<endl;
        cout<<"Remaining Qty : "
<<components[i].getQuantity()<<endl;

        cout<<"\nPurchase Successful!\n"<<endl;
        break;
    }
}

if(!found)
{
    cout<<"\nComponent Not Found!\n"<<endl;
}

}

void Inventory::saveData()
{
    ofstream fout("components.txt");

    if(!fout)
    {
        cout<<"\nUnable to Open File!\n"<<endl;
        return;
    }

    for(int i=0;i<components.size();i++)
    {
        fout<<components[i].getType()<<endl;
        fout<<components[i].getName()<<endl;
    }
}

```

```

        fout<<components[i].getQuantity()<<endl;
        fout<<components[i].getPrice()<<endl;
        fout<<components[i].getRating()<<endl;
        fout<<components[i].getDescription()<<endl;
    }

    fout.close();
    cout<<"\nData Saved Successfully!"<<endl;
}

void Inventory::loadData()
{
    ifstream fin("components.txt");

    if(!fin)
        return;

    components.clear();

    while(true)
    {
        InventoryItem item;

        string type,name,rating,description;
        int quantity;
        float price;

        if(!getline(fin,type))
            break;

        getline(fin,name);
        fin>>quantity;
        fin>>price;
        fin.ignore();
        getline(fin,rating);
        getline(fin,description);
        item.setType(type);
        item.setName(name);
        item.setQuantity(quantity);
        item.setPrice(price);
        item.setRating(rating);
        item.setDescription(description);

        components.push_back(item);
    }

    fin.close();
}

int main()
{
    Inventory store;
    store.loadData();
    int choice;

    do
    {
        store.menu();
        cout<<"\nEnter Choice : ";
        cin>>choice;

        switch(choice)
        {
            case 1:
                store.addComponent();
                break;
            case 2:
                store.viewComponent();
                break;
            case 3:

```

```

        store.updateComponent();
        break;
    case 4:
        store.deleteComponent();
        break;
    case 5:
        store.purchaseComponent();
        break;
    case 6:
        store.saveData();
        break;
    case 7:
        store.saveData();
        cout<<"\nThank You!"<<endl;
        break;

    default:
        cout<<"\nInvalid Choice!\n"<<endl;
}

}while(choice!=7);

return 0;
}

```

CHAPTER 4: RESULTS AND ANALYSIS

4.1 Gantt Chart

The following Gantt chart shows the approximate time duration allotted to each phase of the project.

Table 4.1: Gantt Chart of Project Schedule

Task	Week 1	Week 2	Week 3	Week 4	Week 5
Requirement Analysis	✓				
Class Design	✓	✓			
Coding of Modules		✓	✓		
File Handling Integration			✓		
Testing and Debugging				✓	
Report Writing				✓	✓

4.2 Results

The program was compiled and executed on the console, and the following results were obtained after testing the Add, View, and Purchase modules with sample data.

4.2.1 Add component

```
ELECTRONICS COMPONENTS STORE MANAGEMENT

1. Add Component
2. View Component
3. Update Component
4. Delete Component
5. Purchase Component
6. Save Data
7. Exit

Enter Choice : 1
Select Component Type
1. Resistor
2. Capacitor
3. Inductor
4. Diode
5. LED
6. Transistor
7. IC
8. Microcontroller
9. Display Module

Enter Choice :1
Component Name : fixed resistor
Quantity : 10
Unit Price : 200
Power Rating / Voltage : 10V
Description : Carbon film resistor

Component Added Successfully!
```

4.2.2 View component

Component Added Successfully!

ELECTRONICS COMPONENTS STORE MANAGEMENT

1. Add Component
2. View Component
3. Update Component
4. Delete Component
5. Purchase Component
6. Save Data
7. Exit

Enter Choice : 2

Type	Name	Qty	Price	Rating	Description
Resistor	fixed resistor	10	200	10V	Carbon film resistor

4.2.3.Update component

ELECTRONICS COMPONENTS STORE MANAGEMENT

1. Add Component
2. View Component
3. Update Component
4. Delete Component
5. Purchase Component
6. Save Data
7. Exit

Enter Choice : 3

Enter Component Name to Update : fixed resistor

Current Details

Type : Resistor
Name : fixed resistor
Quantity : 10
Unit Price : 200
Rating : 10V
Description : Carbon film resistor

Enter New Details

Component Name : variable resistor
Quantity : 20
Unit Price : 100
Power Rating / Voltage : 5V
Description : adjustable resistance value

Component Updated Successfully!

4.2.4.Purchase component

```
ELECTRONICS COMPONENTS STORE MANAGEMENT

1. Add Component
2. View Component
3. Update Component
4. Delete Component
5. Purchase Component
6. Save Data
7. Exit

Enter Choice : 5

Enter Component Name to Purchase : variable resistor

Available Stock : 20
Enter Quantity to Purchase : 11
PURCHASE BILLComponent Name : variable resistor
Component Type : Resistor
Purchased Qty : 11
Unit Price : Rs. 100
Total Bill : Rs. 1100
Remaining Qty : 9

Purchase Successful!
```

4.2.5.Save Data

```
ELECTRONICS COMPONENTS STORE MANAGEMENT

1. Add Component
2. View Component
3. Update Component
4. Delete Component
5. Purchase Component
6. Save Data
7. Exit

Enter Choice : 6

Data Saved Successfully!
```

4.2.6.Exit component

```
ELECTRONICS COMPONENTS STORE MANAGEMENT

1. Add Component
2. View Component
3. Update Component
4. Delete Component
5. Purchase Component
6. Save Data
7. Exit

Enter Choice : 7

Data Saved Successfully!

Thank You!
```

These results confirm that the modules for adding, viewing,,updating,deleting,purchasing, saving and exit components work correctly as per the algorithms described in Chapter 3. The saveData() and loadData() functions were also verified to correctly write and read back the component records from components.txt, confirming that the inventory data persists correctly between successive runs of the program.

CHAPTER 5: CONCLUSION, LIMITATIONS AND FUTURE ENHANCEMENT

5.1 Conclusion

The Electronics Component Store Management System successfully demonstrates the use of Object Oriented Programming concepts, including classes, public inheritance, virtual functions and runtime polymorphism, encapsulation using getter and setter functions, and file handling, to build a working console-based inventory management application. The system allows a user to add, view, update, delete, and purchase electronic components such as resistors, capacitors, inductors, diodes, LEDs, transistors, ICs, microcontrollers, and display modules, and it maintains the inventory data persistently in a text file, components.txt, so that records are not lost when the program is closed. Testing of the program confirmed that all the implemented modules, namely add, view, update, delete, purchase, save, and load, function correctly for the sample data used.

5.2 Limitations

- The system uses linear search by component name for update, delete, and purchase operations, which becomes slower as the number of components grows large.
- Data is stored in a plain text file, components.txt, without any encryption, so it is not secure against unauthorized access or accidental corruption.
- The application is entirely console based and does not provide a graphical user interface.
- There is no separate login or user authentication system, so anyone with access to the program can modify the inventory.
- The system does not currently generate reports such as low-stock alerts or sales summaries over a period of time.

5.3 Future Enhancement

Some possible future enhancements of the project are listed below:

- Add a graphical user interface using a framework such as Qt or a web-based front end to make the system more user friendly.
- Replace the text file storage with a proper database system such as SQLite or MySQL for more reliable and secure data storage.
- Implement a login and user authentication module to restrict access to authorized users only.
- Add a low-stock alert feature that automatically notifies the store owner when the quantity of a component falls below a set threshold.

REFERENCES

- [1] Santosh Giri and Basanta Joshi, “Transfer learning-based image visualization using CNN,” *International Journal of Artificial Intelligence & Applications*, vol. 10, no. 4, pp. 47–55, 2019.
- [2] B. Stroustrup, *The C++ Programming Language*, 4th ed. Boston, MA, USA: Addison-Wesley, 2013.
- [3] E. Hart, *Doing a Literature Review: Releasing the Social Science Research Imagination*. London, U.K.: Sage Publications, 1998.
- [3] M. Shaw, “Comparing architectural design styles,” *IEEE Software*, vol. 12, no. 6, pp. 27–41, Nov. 1995.